

Question 1

Write an SQL query to display the names and salaries of employees whose salary is greater than the average salary in the company.

```
INSERT 0 0
postgres=# select * from employees;
 id | name  | salary | department | commission | job_role
-----+-----+-----+-----+-----+-----
  1 | Aman  | 50000 | IT          |             | Developer
  2 | Riya  | 60000 | HR          |          50 | Manager
  3 | Rahul | 45000 | IT          |             | Developer
  4 | Sneha | 70000 | Finance    |          70 | Analyst
  5 | Karan | 55000 | IT          |          30 | Tester
  6 | Meena | 80000 | HR          |             | Manager
(6 rows)

postgres=# SELECT name, salary
postgres=# FROM employees
postgres=# WHERE salary > (SELECT AVG(salary) FROM employees);
 name | salary
-----+-----
 Sneha | 70000
 Meena | 80000
(2 rows)
```

Question 2

2. Write a query to retrieve the top 5 highest-paid employees from an employees table.

Explain how sorting affects the output.

```
postgres=# SELECT *
postgres=# FROM employees
postgres=# ORDER BY salary DESC
postgres=# LIMIT 5;
 id | name  | salary | department | commission | job_role
-----+-----+-----+-----+-----+-----
  6 | Meena | 80000 | HR          |             | Manager
  4 | Sneha | 70000 | Finance    |          70 | Analyst
  2 | Riya  | 60000 | HR          |          50 | Manager
  5 | Karan | 55000 | IT          |          30 | Tester
  1 | Aman  | 50000 | IT          |             | Developer
(5 rows)

postgres=#
```

3. Write a query to calculate:

- Total number of employees
- Average salary
- Minimum and maximum salary

Explain the difference between aggregate and scalar functions.

```
postgres=# SELECT
postgres=# COUNT(*) AS total_employees,
postgres=# AVG(salary) AS average_salary,
postgres=# MIN(salary) AS minimum_salary,
postgres=# MAX(salary) AS maximum_salary
postgres=# FROM employees;
 total_employees | average_salary | minimum_salary | maximum_salary
-----+-----+-----+-----
(1 row)
```

4. Given a sales table with columns (region, amount), write a query to find total sales per region.

Filter only those regions where total sales exceed 50,000.

```
postgres=# select * from sales;
 region | amount
-----+-----
 North  | 20000
 South  | 40000
 East   | 30000
 West   | 60000
 North  | 35000
 South  | 20000
 East   | 25000
 West   | 10000
(8 rows)

postgres=# SELECT region, SUM(amount) AS total_sales
postgres=# FROM sales
postgres=# GROUP BY region
postgres=# HAVING SUM(amount) > 50000;
 region | total_sales
-----+-----
 East   | 55000
 South  | 60000
 West   | 70000
 North  | 55000
(4 rows)

postgres=#
```

5. Write a query to find the number of unique job roles in an employees table. Explain why DISTINCT is necessary here.

```

postgres=# SELECT COUNT(DISTINCT job_role) AS unique_roles
postgres=# FROM employees;
 unique_roles
-----
          4
(1 row)

```

6. Write a query to retrieve students who scored between 60 and 80 marks.

Rewrite the same query using BETWEEN.

```

postgres=# select * from students;
 id | name  | marks
----+-----+-----
  1 | Aman  |    55
  2 | Riya  |    65
  3 | Rahul |    70
  4 | Sneha |    80
  5 | Karan |    45
  6 | Meena |    75
(6 rows)

postgres=# SELECT *
postgres=# FROM students
postgres=# WHERE marks BETWEEN 60 AND 80;
 id | name  | marks
----+-----+-----
  2 | Riya  |    65
  3 | Rahul |    70
  4 | Sneha |    80
  6 | Meena |    75
(4 rows)

```

7. Write a query to display employees whose commission is NULL. Explain the correct way to check NULL values in SQL.

```

postgres=# SELECT *
postgres=# FROM employees
postgres=# WHERE commission IS NULL;
 id | name  | salary | department | commission | job_role
-----+-----+-----+-----+-----+-----
  1 | Aman  | 500000 | IT          |             | Developer
  3 | Rahul | 450000 | IT          |             | Developer
  6 | Meena | 800000 | HR          |             | Manager
(3 rows)

postgres=#

```

8. Write a query to increase the salary of employees in the "IT" department by 10%.

Explain how arithmetic operations are handled in SQL.

```

postgres=# UPDATE employees
postgres=# SET salary = salary * 1.10
postgres=# WHERE department = 'IT';
UPDATE 3
postgres=#

```

9. Write a query to delete records of students who scored less than 40 marks.

What precaution should be taken before executing DELETE?

```

postgres=# DELETE FROM students
postgres=# WHERE marks < 40;
DELETE 0
postgres=#

```

10. Write a query to find employees who earn more than the average salary of their department (without using joins).

Explain the logic of the subquery.

```
postgres=# SELECT name, salary, department
postgres=# FROM employees e
postgres=# WHERE salary > (
postgres=#     SELECT AVG(salary)
postgres=#     FROM employees
postgres=#     WHERE department = e.department
postgres=# );
 name | salary | department 
-----+-----+-----
Meena | 80000  | HR
Karan | 60500  | IT
(2 rows)

postgres=#
```